

LookBack Blockchain System - Detailed Technical Guide

1. Purpose and Scope

This document explains the complete blockchain subsystem implemented in LookBack:

- Smart contract behavior and storage model
- Wallet and network connection flow (MetaMask + Ganache)
- Hashing and integrity verification logic
- Firestore to blockchain data linkage
- UI integration points (scanner, vault, search)
- Deployment model and operational runbook
- Security, risks, and practical recommendations

The guide is based on the current project code.

2. High-Level Architecture

LookBack uses a hybrid architecture:

1. Off-chain evidence metadata and AI outputs are stored in Firebase/Firestore.
2. A deterministic SHA-256 hash of each evidence image is stored on-chain.
3. The blockchain record acts as an immutable proof anchor.
4. Verification compares three representations when available:
 - Fresh hash from current image
 - Hash saved in Firebase
 - Hash stored on blockchain

Result: the system can detect tampering, inconsistency, or drift across storage layers.

3. Smart Contract Deep Dive

Contract file:

- contracts/EvidenceStorage.sol

3.1 Data Model

The contract defines:

- EvidenceRecord struct:
 - evidenceId: app-level evidence identifier
 - evidenceHash: SHA-256 hex hash
 - timestamp: block timestamp when stored
 - officerAddress: msg.sender who stored the record

Storage mappings:

- evidenceRecords: maps evidenceId to EvidenceRecord
- evidenceExists: existence guard for uniqueness

3.2 Event Model

Event emitted on write:

- EvidenceStored(evidenceId, evidenceHash, timestamp, officerAddress)

This event creates an immutable audit log in transaction receipts.

3.3 Function Behavior

createEvidence(_evidenceId, _evidenceHash)

- Rejects duplicate evidence IDs via evidenceExists guard.
- Writes record and marks existence.
- Emits EvidenceStored event.

getEvidence(_evidenceId)

- Reverts if evidence does not exist.
- Returns the full record fields.

verifyEvidence(_evidenceId, _hashToVerify)

- Returns false if no record exists.
- Compares keccak256(encoded stored hash) with keccak256(encoded provided hash).
- Returns boolean integrity match.

4. Client Blockchain Service Design

Service file:

- services/BlockchainService.ts

4.1 Core Responsibilities

- Connect browser wallet provider (MetaMask)
- Enforce expected local Ganache network
- Build typed Ethers contract instance
- Hash image data in browser-compatible way
- Write evidence hash on-chain
- Read record and verify integrity

4.2 Contract Configuration

The service loads:

- CONTRACT_ADDRESS from BLOCKCHAIN_CONTRACT_ADDRESS or fallback
- CONTRACT_ABI embedded in the service

It also defines network expectations:

- GANACHE_CHAIN_ID default 0x539
- GANACHE_RPC_URL default <http://127.0.0.1:7545>

4.3 Wallet and Network Control

The service first checks for window.ethereum and then:

1. Reads current chain
2. Switches chain if needed using wallet_switchEthereumChain

3. Adds chain if missing using `wallet_addEthereumChain`
4. Requests account access with `eth_requestAccounts`
5. Creates `BrowserProvider` and signer

This ensures contract calls run on the intended local chain.

4.4 Safety Guards Added

Before contract calls, the service validates:

1. Contract address is not equal to signer address
 - Prevents sending contract calldata to a normal wallet address
2. Contract code exists at contract address
 - `provider.getCode(address)` must not return `0x`

If validation fails, a clear operational error is thrown.

5. Hashing and Integrity Logic

5.1 Hash Generation

Functions:

- `hashDataUrl(dataUrl)`
- `hashFile(file)`
- `hashImageUrl(imageUrl)`

Normalization:

- `trim`
- `lowercase`
- `remove 0x prefix`

This avoids false mismatch due to encoding or prefix differences.

5.2 Verification Decision Matrix

`verifyEvidenceRecord` returns:

- `blockchainMatchesFreshHash`
- `blockchainMatchesFirebaseHash`
- `firebaseMatchesFreshHash`

Interpretation:

- Verified: `blockchain == fresh image hash`
- Warning A: `blockchain differs` but `Firebase == fresh`
- Warning B: `blockchain == Firebase` but `fresh differs`
- Mismatch: all differ or critical mismatch

This gives forensic context rather than a single yes/no.

6. Data Flow Across the App

6.1 Ingestion and On-Chain Storage

Scanner file:

- screens/DocumentScanner.tsx

Flow:

1. Capture/upload image
2. Compute fileHash via hashDataUrl
3. Save evidence metadata in Firestore first
4. Call createEvidenceOnChain(evidenceId, fileHash)
5. On success, update Firestore with:
 - blockchainTxHash
 - blockchainStored = true
 - blockchainEvidenceId

The app continues even if blockchain write fails, so ingestion remains resilient.

6.2 Vault Verification UI

Vault file:

- screens/EvidenceVault.tsx

Capabilities:

- Manual Verify Integrity action
- Status labels: Verified, Warning, Mismatch
- Proof details shown in modal:
 - on-chain timestamp
 - officer address
 - chain hash snippet
 - configured contract address

6.3 Search Verification UI

Search file:

- screens/EvidenceSearch.tsx

Capabilities added:

- Verify Integrity action in detail modal
- Uses linked evidenceId from processed document
- Fetches evidence metadata from Firestore by ID
- Runs same verifyEvidenceRecord comparison logic
- Displays verification status and proof block

7. Firestore Linkage Model

Service file:

- services/FirebaseService.ts

Key fields used for blockchain integration in evidence documents:

- fileHash
- blockchainTxHash
- blockchainStored

- blockchainEvidenceId

Processed document records link back using:

- evidenceId

A helper exists to fetch evidence directly:

- getEvidenceById(id)

This enables verification from views that start from processed docs instead of raw evidence items.

8. Deployment Model and Current Local Chain Setup

Hardhat config:

- hardhat.config.js

Deploy script:

- scripts/deploy.js

Current local setup assumptions:

- Ganache RPC at <http://127.0.0.1:7545>
- Chain ID expected as 1337 (0x539)
- Contract artifacts read from Hardhat artifacts folder

Deployment output:

- scripts/contract-address.txt stores latest deployed contract address

Operational rule:

- The app contract address must match the chain currently selected in MetaMask.
- If Ganache is reset/restarted with a fresh chain state, redeploy and update address.

9. Common Errors and Root Causes

Error: External transactions to internal accounts cannot include data

Cause:

- Contract address was set to a wallet (EOA) instead of a deployed contract.

Fix:

- Set BLOCKCHAIN_CONTRACT_ADDRESS to deployed contract address.

Error: No contract deployed at

Cause:

- Wrong network selected in MetaMask, or wrong contract address for the active chain.

Fix:

- Ensure MetaMask chain is Ganache 0x539.
- Verify provider.getCode(address) is not 0x.

Error: insufficient funds for intrinsic transaction cost during deploy

Cause:

- Deployment signed with private key that does not match funded Ganache account.

Fix:

- Use private key of funded account from current Ganache workspace.

10. Security and Operational Notes

10.1 Good Practices

- Keep contract address in environment variables per environment.
- Keep chain ID and RPC configurable per environment.
- Keep ABI and contract source synchronized.
- Log and monitor transaction hashes for audit trails.

10.2 Local Dev Risks Identified

- Private keys are currently hardcoded in blockchain tool config files.
- This is acceptable only for local-only development.
- Move private keys to environment variables before any shared or production workflow.

10.3 Data Integrity Limits

- Blockchain secures hash proof, not file content availability.
- If source file is lost, blockchain still proves prior hash but cannot recover content.
- Consider durable storage strategy plus retention policy.

11. Practical Verification Procedure

1. Confirm Ganache is running on expected RPC.
2. Confirm MetaMask is connected to the same chain ID.
3. Confirm contract code exists at configured CONTRACT_ADDRESS.
4. Upload evidence and approve transaction.
5. Verify in Evidence Vault and/or Evidence Search.
6. Cross-check Firestore fields and transaction hash.

12. Suggested Next Improvements

1. Add explorer-like local transaction detail links in UI.
2. Add automatic retry queue for failed blockchain writes.
3. Add contract version field in app config to prevent ABI drift.
4. Add chain mismatch banner globally in the app header.
5. Migrate private keys to environment-based secret loading.

13. File Index (Blockchain-Relevant)

Core contract:

- contracts/EvidenceStorage.sol

Client blockchain service:

- services/BlockchainService.ts

Ingestion + chain write:

- screens/DocumentScanner.tsx

Evidence verification UI:

- screens/EvidenceVault.tsx
- screens/EvidenceSearch.tsx

Firestore service and evidence linkage:

- services/FirebaseService.ts

Deployment and chain config:

- hardhat.config.js
- scripts/deploy.js
- scripts/contract-address.txt

Reference docs:

- BLOCKCHAIN_SETUP.md
- BLOCKCHAIN_TESTING_GUIDE.md
- BLOCKCHAIN_ENHANCEMENTS.md